

Test Documentation — Pollinator Habitat

This document explains every test file in the project, covering both the **backend** and **frontend**. Each test is described line-by-line (or block-by-block where lines are part of the same logical unit).

- **Test framework:** Vitest (both frontend and backend)
 - **Backend integration helper:** Supertest
 - **Frontend render helper:** React Testing Library + `@testing-library/user-event`
-

Running Tests

All commands run from the `Pollinator-Habitat/` directory (the monorepo root with `package.json`).

Run all backend tests

```
npm run test:backend
```

Run all portal tests

```
npm run test:portal
```

Run a single test file

```
# From within backend/  
npx vitest run tests/unit/campbell/game.service.test.ts  
  
# From within portal/  
npx vitest run src/__tests__/SomeComponent.test.tsx
```

Run with coverage

```
# Backend  
cd backend && npx vitest run --coverage  
  
# Portal  
cd portal && npx vitest run --coverage
```

Watch mode (re-runs on file change)

```
cd backend && npx vitest
```

Table of Contents

Backend (`/Pollinator-Habitat/backend/tests/`)

1. [CheckSession.service.test.ts](#)
2. [api.test.ts](#) 2b. [api.jwt-guard.test.ts](#)
3. [api.utilities.test.ts](#)
4. [getOrCreatePlayerSession.test.ts](#)
5. [index.test.ts](#)
6. [prisma.test.ts](#) 6b. [middleware.test.ts](#)
7. [routes.test.ts \(integration\)](#)
8. [databaseOperations/getChildrenAndAdultBySession.test.ts](#)
9. [databaseOperations/getChildNumAndAdultDateToForever.test.ts](#)
10. [databaseOperations/getActiveRouteForPlayerSession.test.ts](#)
11. [databaseOperations/hasSession.test.ts](#)
12. [databaseOperations/getOrCreatePlayerSession.test.ts](#)
13. [unit/campbell/game.service.test.ts](#)
14. [unit/campbell/route.service.test.ts](#)
15. [unit/campbell/reshape.utilities.test.ts](#) 34b.
[unit/campbell/getNextRouteForPlayerSession.service.test.ts](#)

Frontend (/Pollinator-Habitat/frontend/testing/)

13. [AccessibilityPage.test.tsx](#)
14. [HomePage.test.tsx](#)
15. [Layout.test.tsx](#)
16. [Middleware.test.tsx](#)
17. [PollinatorCollectionPage.test.tsx](#)
18. [RedirectPage.test.tsx](#)
19. [RootPage.test.tsx](#)
20. [RootPageAPI.test.tsx](#)
21. [RoutePageFlow.test.tsx](#)
22. [jwtService.test.tsx](#)
23. [routeService.test.tsx](#)
24. [surveyStorageService.test.tsx](#)
25. [UAT/OneTimePartyASizeSurvey.test.tsx](#)
26. [UAT/PollinatorCollection.test.tsx](#)
27. [UAT/RepeatableRoutePlay.test.tsx](#)

Portal (/Pollinator-Habitat/portal/testing/)

28. [surveyPortalStructure.test.tsx](#)
29. [surveyPortalFunction.test.tsx](#)
30. [inputValidator.test.tsx](#)
31. [layout.test.tsx](#)
32. [exportTool.test.tsx](#)
33. [portalMenu.test.tsx](#) (#24-portalmenutest tsx)
34. [splashPage.test.tsx](#)
35. [UAT/ExportSearchResults.test.tsx](#)

Backend Tests

1. CheckSession.service.test.ts

Path: [backend/tests/unit/campbell/CheckSession.service.test.ts](#) **Purpose:** Unit-tests the `checkSessionService.requireSession()` function, which guards game logic by verifying a session

exists in the database before proceeding.

```
import { test, expect, vi, beforeEach } from "vitest";
```

Imports the core Vitest utilities: `test` to define a test case, `expect` for assertions, `vi` for mocking, and `beforeEach` for setup hooks.

```
vi.mock("../../src/services/db/DatabaseOperationsServices/Read/hassession.service",  
() => ({  
  HasSessionService: {  
    hasSession: vi.fn(),  
  },  
}));
```

Replaces the real `hassession.service` module with a mock. The exported `HasSessionService.hasSession` is replaced by a `vi.fn()` stub so the test controls what it returns without touching the database.

```
import { checkSessionService } from  
"../../src/services/gameServiceUtils/CheckSession.service";  
import { HasSessionService } from  
"../../src/services/db/DatabaseOperationsServices/Read/hassession.service";
```

Imports the module under test (`checkSessionService`) and the mocked dependency (`HasSessionService`) so the test can configure the stub and assert it was called.

```
beforeEach(() => {  
  vi.resetAllMocks();  
});
```

Before every test, resets all mocks back to their initial state (clears call history and removes any custom implementations). This prevents state leaking from one test into the next.

Test 1 — "resolves when session exists"

```
test("resolves when session exists", async () => {
```

Declares an async test case: the function under test is async so the test must await its result.

```
vi.mocked(HasSessionService.hasSession).mockResolvedValue(true);
```

Configures the mock to return a resolved promise with `true` , simulating a database lookup that finds the session.

```
await expect(checkSessionService.requireSession(1)).resolves.toBeUndefined();
```

Asserts that calling `requireSession(1)` resolves (does not throw) and that its resolved value is `undefined` — i.e., it succeeds silently.

```
expect(HasSessionService.hasSession).toHaveBeenCalled(1);
```

Asserts that the mock was called with the exact session ID `1`, confirming the service passes the argument through correctly.

Test 2 — "throws when session does not exist"

```
test("throws when session does not exist", async () => {  
  vi.mocked(HasSessionService.hasSession).mockResolvedValue(false);
```

Configures the mock to return `false`, simulating a session that is not found in the database.

```
  await expect(checkSessionService.requireSession(999)).rejects.toThrow("Invalid  
  session ID");
```

Asserts that `requireSession(999)` rejects (throws) with the error message `"Invalid session ID"`.

```
  expect(HasSessionService.hasSession).toHaveBeenCalled(999);
```

Confirms that the service forwarded the session ID `999` to `hasSession`.

2. api.test.ts

Path: `backend/tests/unit/campbell/api.test.ts` **Purpose:** Unit-tests the three Express request-handler functions exported from `api.ts`: `Validate`, `startNewGame`, and `completeRoute`. All dependencies (database, JWT, game service) are mocked so each handler is tested in isolation.

```
import { test, expect, vi, beforeEach } from "vitest";
```

Standard Vitest imports.

```
const { mockGameService } = vi.hoisted(() => ({  
  mockGameService: {  
    startOrJoin: vi.fn(),  
    completeRouteAndAdvance: vi.fn(),  
  },  
}));
```

`vi.hoisted` runs before module mocks are applied (before any `vi.mock` calls), so this creates the `mockGameService` object early. This is necessary because the `vi.mock` factory for `game.service` below needs to reference `mockGameService`.

```
vi.mock("../../../src/services/db/prisma", () => ({ prisma: {} }));
```

Mocks the Prisma client module with an empty object so no real database connections are made.

```
vi.mock("../../src/services/game.service", () => ({
  GameService: vi.fn(function () { return mockGameService; }),
}));
```

Mocks `GameService` as a constructor that returns `mockGameService`. When the handler calls `new GameService(...)`, it gets back the mock object with stubbed `startOrJoin` and `completeRouteAndAdvance`.

```
vi.mock("jsonwebtoken", () => ({
  default: { sign: vi.fn(() => "mock-token") },
}));
```

Mocks the JWT library so token signing is controlled. No real tokens are generated.

```
vi.mock("../../src/services/db/DatabaseOperationsServices/Read/hassession.service",
() => ({
  HasSessionService: { hasSession: vi.fn() },
}));
vi.mock("../../src/api/Utilities/api.utilities", () => ({
  APIUtilities: {
    toNumber: vi.fn(),
    generateUniquePlayerId: vi.fn(),
    getSessionIdFromReq: vi.fn(),
    getPlayerIdFromReq: vi.fn(),
  },
}));
vi.mock("../../src/api/Utilities/reshape.Utilites", () => ({
  ReshapeUtilities: { mapRouteDTOToLegacyRoute: vi.fn((r) => r) },
}));
```

Mocks the remaining dependencies: session lookup, utility functions for extracting IDs from requests, and a reshape utility that transforms route data into legacy format.

```
import { Validate, startNewGame, completeRoute } from "../../src/api/api";
import { HasSessionService } from "...";
import { APIUtilities } from "...";
import { ReshapeUtilities } from "...";
import jwt from "jsonwebtoken";
```

Imports the three handlers under test and the mocked dependencies so their stubs can be configured.

```
function mockRes() {
  const res: any = {};
  res.status = vi.fn().mockReturnValue(res);
  res.json = vi.fn().mockReturnValue(res);
}
```

```
    return res;
}
```

A factory function that creates a minimal mock Express `Response` object. Both `.status()` and `.json()` return `res` itself (chaining), mimicking `res.status(200).json({...})` .

```
beforeEach(() => {
  vi.clearAllMocks();
});
```

Clears all mock call history before each test.

Validate handler tests

Test — "Validate: returns 400 if sessionId is null"

```
vi.mocked(APIUtilities.toNumber).mockReturnValue(null);
```

Simulates `toNumber` failing to parse the session ID (returns `null`).

```
const req: any = { body: { sessionId: "bad" } };
const res = mockRes();
await Validate(req, res);
expect(res.status).toHaveBeenCalledWith(400);
expect(res.json).toHaveBeenCalledWith({ error: "Session ID is required" });
```

Calls the handler with a bad session ID and asserts a `400` response with the correct error body.

Test — "Validate: returns 401 if session does not exist"

```
vi.mocked(APIUtilities.toNumber).mockReturnValue(42);
vi.mocked(HasSessionService.hasSession).mockResolvedValue(false);
```

Makes `toNumber` succeed (returns `42`) but `hasSession` return `false` .

```
expect(res.status).toHaveBeenCalledWith(401);
expect(res.json).toHaveBeenCalledWith({ error: "Invalid session ID" });
```

Asserts the handler returns `401 Unauthorized` .

Test — "Validate: returns 200 with token on success"

```
vi.mocked(APIUtilities.toNumber).mockReturnValue(42);
vi.mocked(HasSessionService.hasSession).mockResolvedValue(true);
vi.mocked(APIUtilities.generateUniquePlayerId).mockResolvedValue(99);
vi.mocked(jwt.sign).mockReturnValue("mock-token" as any);
```

All preconditions succeed: valid session ID, session exists, unique player ID generated, JWT signed.

```
expect(res.status).toHaveBeenCalledWith(200);
expect(res.json).toHaveBeenCalledWith({ token: "mock-token" });
```

Asserts the handler returns `200` with a token.

Test — "Validate: returns 500 on unexpected error"

```
vi.mocked(HasSessionService.hasSession).mockRejectedValue(new Error("DB down"));
```

Simulates a database crash.

```
expect(res.status).toHaveBeenCalledWith(500);
expect(res.json).toHaveBeenCalledWith(
  expect.objectContaining({ error: "Wrong input", message: "DB down" })
);
```

Asserts a `500` response with both an `error` and a `message` field. `objectContaining` means the response may have extra fields.

startNewGame handler tests

Test — "returns 400 if sessionId is null" `getSessionIdFromReq` returns `null` → handler returns `400`.

Test — "returns 200 on success"

```
vi.mocked(APIUtilities.getSessionIdFromReq).mockReturnValue(1);
vi.mocked(APIUtilities.getPlayerIdFromReq).mockReturnValue(2);
mockGameService.startOrJoin.mockResolvedValue({ playerId: 2, activeRoute: { id: 5 } });
```

Stubs valid session and player IDs plus a resolved game state.

```
expect(res.json).toHaveBeenCalledWith({
  sessionId: "1",
  player: { id: 2, routesCompleted: [], activeRoute: { id: 5 } },
  availableRoutes: [],
});
```

Asserts the response shape matches the expected contract.

Test — "returns 200 with null activeRoute when none returned" Same setup but `activeRoute: null` from the service — asserts the handler passes `null` through correctly.

Test — "returns 500 on unexpected error" `startOrJoin` rejects → `500` response.

Test — "returns 500 with unknown message when non-Error thrown"

```
mockGameService.startOrJoin.mockRejectedValue("string error");
```

Throws a plain string instead of an `Error` object.

```
expect(res.json).toHaveBeenCalledWith(
  expect.objectContaining({ message: "Unknown error" })
);
```

Asserts the handler has a fallback for non-`Error` rejections.

`completeRoute` handler tests

Test — "returns 400 if sessionId is null" Same pattern as `startNewGame` .

Test — "returns 404 on 'No active route to complete'"

```
mockGameService.completeRouteAndAdvance.mockRejectedValue(
  new Error("No active route to complete")
);
expect(res.status).toHaveBeenCalledWith(404);
```

The handler differentiates this specific domain error and returns `404` instead of `500` .

Test — "returns 200 with new route on success"

```
mockGameService.completeRouteAndAdvance.mockResolvedValue({
  newActiveRoute: newRoute,
  routesCompleted: 3,
  completedPollinators: ["bee", "butterfly"],
});
```

Full success path — asserts `200` with `success: true` , `message` , `newActiveRoute` , `routesCompleted` , and `completedPollinators` .

Test — "returns 500 on unexpected error" Generic database crash → `500` .

Test — "returns 200 with null newActiveRoute when none returned" No more routes to assign → `newActiveRoute: null` is passed through in `200` response.

Test — "passes completedPollinators through from service"

```
const pollinators = ["monarch", "honeybee", "bumblebee"];
expect(res.json).toHaveBeenCalledWith(
  expect.objectContaining({ completedPollinators: pollinators })
);
```

Verifies the handler faithfully relays the pollinator array without modification.

Test — "Validate: uses JWT_SECRET env var when set"

```
expect(jwt.sign).toHaveBeenCalledWith(expect.anything(), expect.any(String),
expect.anything());
```



```
expect(res.status).toHaveBeenCalledWith(200);
```

Confirms that `jwt.sign` is called with a string secret. The test no longer sets/deletes `process.env.JWT_SECRET` manually — `JWT_SECRET` is now injected globally via `vitest.config.ts` (`env: { JWT_SECRET: 'test-secret-not-for-production' }`), so the assertion checks `expect.any(String)` rather than a specific value.

Test — "Validate: returns 500 with 'Unknown error' when non-Error thrown" Mirrors the `startNewGame` non-Error rejection test for the `Validate` handler.

Test — "completeRoute: returns 500 with String(error) when non-Error thrown"

```
mockGameService.completeRouteAndAdvance.mockRejectedValue("plain string error");
expect(res.json).toHaveBeenCalledWith(
  expect.objectContaining({ message: "plain string error" })
);
```

When a raw string is thrown, the handler converts it via `String()` and includes it in the `500` response.

2b. api.jwt-guard.test.ts

Path: [backend/tests/unit/campbell/api.jwt-guard.test.ts](#) **Purpose:** Tests the startup guard in `api.ts` that throws if `JWT_SECRET` is not set. Must run in an isolated module context (no other test in this suite may set `JWT_SECRET` before this file loads `api.ts`).

```
vi.mock("../src/services/db/prisma", () => ({ prisma: {} }));
vi.mock("../src/services/game.service", () => ({ GameService: vi.fn(function
() { return {}; }) }));
vi.mock("jsonwebtoken", () => ({ default: { sign: vi.fn() } }));
// ... (all other heavy imports mocked)
```

Every direct dependency of `api.ts` is mocked so the module can be imported without triggering real DB connections, JWT signing, or any of the 20+ statistics services. This prevents side effects and keeps the test fast.

Test — "api.ts throws when JWT_SECRET is not set"

```
test("api.ts throws when JWT_SECRET is not set", async () => {
  const original = process.env.JWT_SECRET;
  delete process.env.JWT_SECRET;
  try {
    await expect(() => import("../src/api/api")).rejects.toThrow(
      "JWT_SECRET environment variable is required"
    );
  } finally {
    if (original !== undefined) process.env.JWT_SECRET = original;
  }
});
```

1. Saves and deletes `JWT_SECRET` from the environment.

2. Dynamically imports `api.ts` — this triggers the module-level guard (`if (!JWT_SECRET) throw ...`).
3. Asserts the import rejects with the exact error message.
4. `finally` block restores the original value so other tests in the suite are not affected.

The test covers the invariant that the server cannot start (and `api.ts` cannot be loaded) without a JWT signing key. This prevents silent production misconfiguration.

3. `api.utilities.test.ts`

Path: [backend/tests/unit/campbell/api.utilities.test.ts](#) **Purpose:** Unit-tests `APIUtilities` — a set of helper functions for parsing request values and generating unique player IDs.

```
const { mockFindUnique } = vi.hoisted(() => ({
  mockFindUnique: vi.fn(),
}));
vi.mock("../../../src/services/db/prisma", () => ({
  prisma: { playerSession: { findUnique: mockFindUnique } },
}));
```

Hoists a mock for `prisma.playerSession.findUnique` so it can be referenced in the Prisma mock factory. This avoids real DB calls during the `generateUniquePlayerId` tests.

`toNumber` tests

"returns the number if given a finite number" Confirms `42`, `0`, and `-7.5` all pass through unchanged.

"returns null for Infinity" `Infinity` and `-Infinity` are not valid game IDs — must return `null`.

"returns null for NaN" `NaN` is not a number — returns `null`.

"parses a numeric string" `"42"` → `42`, `"3.14"` → `3.14`, `"-1"` → `-1`.

"returns null for an empty or whitespace string" `""` and `" "` are not parseable — return `null`.

"returns null for a non-numeric string" `"abc"` and `"12abc"` cannot be fully parsed — return `null`.

"returns null for null and undefined" Boundary check: `null` and `undefined` inputs return `null`.

`getSessionIdFromReq` tests

"returns sessionId from auth"

```
const req = { auth: { sessionId: 1 } };
expect(APIUtilities.getSessionIdFromReq(req)).toBe(1);
```

Reads `sessionId` from `req.auth` (the decoded JWT payload). Returns the numeric value.

"returns null when auth is absent"

```
const req = { query: { sessionId: "5" }, params: { sessionId: "6" }, body: {  
  sessionId: 7 } };  
expect(APIUtilities.getSessionIdFromReq(req)).toBeNull();
```

When `req.auth` is not present, returns `null` even if `query`, `params`, or `body` contain a `sessionId`. The function is auth-only.

"returns null when auth.sessionId is absent" `req.auth` exists but has no `sessionId` key → returns `null`.

"returns null for an invalid auth.sessionId" `req.auth.sessionId = "notANumber"` → `toNumber` fails → returns `null`.

getPlayerIdFromReq tests

"returns playerId from auth" `req.auth.playerId` is the only source — returns the numeric value.

"returns undefined when auth is absent" Player ID only comes from a decoded JWT; without `auth`, returns `undefined`.

"returns undefined when auth.playerId is absent" `auth` exists but lacks `playerId` key.

"returns undefined when auth.playerId is non-numeric" `"notANumber"` fails the number parse → `undefined`.

generateUniquePlayerId tests

"returns a candidate when no existing record found"

```
mockFindUnique.mockResolvedValue(null);  
const id = await APIUtilities.generateUniquePlayerId(1);  
expect(typeof id).toBe("number");  
expect(id).toBeGreaterThanOrEqual(1);  
expect(id).toBeLessThan(2_000_000_000);  
expect(mockFindUnique).toHaveBeenCalledTimes(1);
```

On the first attempt, no collision is found, so it returns on the first try.

"retries until a free candidate is found"

```
mockFindUnique  
  .mockResolvedValueOnce({ id: 1 })  
  .mockResolvedValueOnce({ id: 2 })  
  .mockResolvedValueOnce({ id: 3 })  
  .mockResolvedValue(null);
```

Simulates three collisions before a free ID is found on the fourth attempt. Asserts `findUnique` was called four times.

"throws after 300 failed attempts"

```
mockFindUnique.mockResolvedValue({ id: 1 });
await expect(APIUtilities.generateUniquePlayerId(1)).rejects.toThrow(
  "Failed to allocate unique playerId after many attempts"
);
expect(mockFindUnique).toHaveBeenCalledTimes(300);
```

Simulates an infinitely full ID space. The function gives up after 300 retries and throws.

generateSessionId tests

"generates a number in the expected format" 🚩 **FAILING** Confirms the returned value is a number and that converting it to a string produces exactly 10 digits (`/^\d{10}$/`). **Currently fails** because `generateSessionId()` returns a JS number, and months `01 – 09` have a leading zero that is dropped when the number is stored — e.g., `0310268795` becomes `310268795` (9 digits, not 10).

"encodes today's date in the first six digits" 🚩 **FAILING** Reads today's MM, DD, YY values from `new Date()` and asserts that `String(sessionId).substring(0, 2) === mm`, etc. **Currently fails** for the same leading-zero reason — `String(310268795).substring(0, 2)` returns `"31"` instead of the expected month `"03"`.

"random portion is four digits between 0000 and 9999" Extracts `sessionStr.substring(6)` and parses it as an integer. Asserts it's `>= 0` and `<= 9999`.

"different calls within same day have same prefix and may differ in random part" Calls `generateSessionId()` twice and asserts both results share the same 6-character date prefix. If the two strings differ, also confirms the random suffixes differ.

"reuses same id for multiple calls on same day, regenerates next day" Uses `vi.useFakeTimers()` and `vi.setSystemTime()` to fix the clock at March 9, 2026:

- Calls `generateSessionId()` twice on day one → both return the same cached value
- Advances clock to March 10, 2026 → next call returns a different value
- Calls `vi.useRealTimers()` to restore the clock

This verifies the per-day caching behaviour: one session ID is minted per calendar day, then reused.

4. getOrCreatePlayerSession.test.ts

Path: [backend/tests/databaseOperations/getOrCreatePlayerSession.test.ts](#) **Purpose:** Tests the service that finds or creates a `PlayerSession` record in the database using `createMany` (skipDuplicates) + `findUniqueOrThrow`.

```
vi.mock("../../src/services/db/prisma", () => ({
  prisma: { playerSession: { createMany: vi.fn(), findUniqueOrThrow: vi.fn() } },
}));
vi.mock("../../src/services/gameServiceUtils/CheckSession.service", () => ({
  checkSessionService: { requireSession: vi.fn() },
}));
```

Mocks `prisma.playerSession.createMany` and `prisma.playerSession.findUniqueOrThrow` — the two DB calls the service makes — and the session validation guard.

```
beforeEach(() => {
  vi.resetAllMocks();
});
```

Resets mocks before each test.

Test — "creates and returns a new player session when none exists"

```
vi.mocked(prisma.playerSession.createMany).mockResolvedValue({ count: 1 });
vi.mocked(prisma.playerSession.findUniqueOrThrow).mockResolvedValue({ id: 42,
playerId: 7 } as any);
const result = await getOrCreatePlayerSessionService.getOrCreatePlayerSession(1, 7);
expect(result).toEqual({ playerId: 42, playerId: 7 });
expect(prisma.playerSession.createMany).toHaveBeenCalledTimes(1);
expect(prisma.playerSession.findUniqueOrThrow).toHaveBeenCalledTimes(1);
```

`createMany` returns `{ count: 1 }` (one row inserted). `findUniqueOrThrow` fetches and returns it.

Test — "returns existing session without creating a duplicate"

```
vi.mocked(prisma.playerSession.createMany).mockResolvedValue({ count: 0 });
vi.mocked(prisma.playerSession.findUniqueOrThrow).mockResolvedValue({ id: 10,
playerId: 3 } as any);
const result = await getOrCreatePlayerSessionService.getOrCreatePlayerSession(5, 3);
expect(result).toEqual({ playerId: 10, playerId: 3 });
expect(prisma.playerSession.createMany).toHaveBeenCalledTimes(1);
expect(prisma.playerSession.findUniqueOrThrow).toHaveBeenCalledTimes(1);
```

`createMany` returns `{ count: 0 }` (row already existed, `skipDuplicates` silently no-oped).

`findUniqueOrThrow` still fetches the existing row. Both calls happen unconditionally regardless of whether the row pre-existed.

Test — "throws when the session does not exist"

```
vi.mocked(checkSessionService.requireSession).mockRejectedValue(new Error("Invalid
session ID"));
await expect(
  getOrCreatePlayerSessionService.getOrCreatePlayerSession(999, 1)
).rejects.toThrow("Invalid session ID");
```

When the session guard throws, the error propagates before any DB call is made.

Test — "throws when playerId is missing"

```
await expect(
  getOrCreatePlayerSessionService.getOrCreatePlayerSession(1, undefined)
).rejects.toThrow("Missing playerId");
```

Validates that `PlayerIdTypeCheck.requirePlayerId` enforces a required `playerId` argument. This check runs before any DB calls.

5. `index.test.ts`

Path: `backend/tests/unit/campbell/index.test.ts` **Purpose:** Tests the server entry-point (`src/index.ts`) which calls `app.listen()` to start the Express server.

```
vi.mock("../../src/app", () => ({
  default: {
    listen: vi.fn((port: number, host: string, callback?: () => void) => {
      if (callback) callback();
    }),
  },
}));
```

Mocks the Express `app` object. The mock `listen` immediately calls its callback, so the "server started" side effect fires synchronously in tests.

```
beforeEach(() => {
  vi.resetModules();
  delete process.env.PORT;
});
```

`vi.resetModules()` clears the module cache so each test gets a fresh `import` of `src/index` . Deleting `process.env.PORT` ensures a clean environment.

Test — "calls `app.listen` with default port 4000"

```
await import("../../src/index");
const { default: app } = await import("../../src/app");
expect(app.listen).toHaveBeenCalledWith(4000, "0.0.0.0", expect.any(Function));
```

Imports the entry-point (triggering `listen`) then asserts it was called with port `4000` and binding address `0.0.0.0` .

Test — "calls `app.listen` with `PORT` from environment variable"

```
process.env.PORT = "8080";
await import("../../src/index");
expect(app.listen).toHaveBeenCalledWith(8080, "0.0.0.0", expect.any(Function));
```

When `PORT` is set in the environment, the server should use that port instead of the default.

Test — "logs the port when server starts"

```
const consoleSpy = vi.spyOn(console, "log").mockImplementation(() => {});
await import("../../src/index");
```

```
expect(consoleSpy).toHaveBeenCalledWith("Listening on 4000");
consoleSpy.mockRestore();
```

Verifies that the startup log message includes the port number.

6. prisma.test.ts

Path: [backend/tests/unit/campbell/prisma.test.ts](#) **Purpose:** Tests that the Prisma client is exported as a singleton — the same instance is returned on every import.

```
vi.mock("@prisma/client", () => {
  const PrismaClient = vi.fn();
  return { PrismaClient };
});
```

Replaces `PrismaClient` with a no-op constructor function. This prevents real database connections while still letting us assert instantiation behavior.

```
import { PrismaClient } from "@prisma/client";
import { prisma } from "../../../src/services/db/prisma";
```

Imports the mock constructor and the singleton instance under test.

Test — "exports a PrismaClient instance"

```
expect(prisma).toBeInstanceOf(PrismaClient);
```

Confirms that `prisma` was created with `new PrismaClient(...)`.

Test — "is a singleton - same instance on re-import"

```
const { prisma: prisma2 } = await import("../../../src/services/db/prisma");
expect(prisma2).toBe(prisma);
```

Re-imports the module and uses strict equality (`toBe` , not `toEqual`) to confirm the same object reference is returned — singleton pattern holds.

Test — "PrismaClient is only instantiated once"

```
expect(PrismaClient).toHaveBeenCalledTimes(1);
```

Ensures the constructor was called exactly once, regardless of how many times the module is imported.

6b. middleware.test.ts (unit)

Path: [backend/tests/unit/middleware.test.ts](#) **Purpose:** Unit-tests the `authenticateJWT` Express middleware in isolation. All three cases are covered: missing/malformed token, valid token, and invalid/expired token.

```
const { mockJwtVerify } = vi.hoisted(() => ({ mockJwtVerify: vi.fn() }));
vi.mock("jsonwebtoken", () => ({ default: { verify: mockJwtVerify } }));
```

Mocks `jwt.verify` before module import so the real JWT library is never called.

Describe — "authenticateJWT — missing or malformed token" (4 tests) Each test confirms that a missing, non-Bearer, empty, or "Bearer" -with-no-token header returns `401 { error: "Missing token" }` and never calls `next()`.

Describe — "authenticateJWT — valid token" (4 tests)

Test — "calls next() when the token is valid" `mockJwtVerify` returns `{ sessionId: 1, playerId: 5 }`. Asserts `next()` called once and `res.status` never called.

Test — "attaches sessionId and playerId to req.auth" Verifies `req.auth` equals `{ sessionId: 42, playerId: 99 }` after a valid token.

Test — "strips the 'Bearer ' prefix before passing token to jwt.verify"

```
expect(mockJwtVerify).toHaveBeenCalledWith(
  "my.actual.token",
  expect.any(String),
  { algorithms: ["HS512"] }
);
```

Confirms the `Bearer` prefix is stripped and `jwt.verify` is called with the **HS512 algorithm constraint**.

Test — "verifies against the JWT_SECRET env variable when set" Same HS512 assertion — confirms the algorithm options object is always passed regardless of environment.

Describe — "authenticateJWT — invalid token" (4 tests) Each test has `mockJwtVerify` throw an error (invalid signature, expired, malformed, generic). All assert `401 { error: "Invalid token" }`, `next()` not called, and `req.auth` undefined.

7. routes.test.ts (Integration)

Path: [backend/tests/integration/routes.test.ts](#) **Purpose:** Integration tests that mount the real Express app and fire HTTP requests against it using Supertest. The API handlers and JWT middleware are mocked so only the routing layer is tested.

```
vi.mock("../src/api/api", () => ({
  Validate: vi.fn((req, res) => res.status(200).json({ token: "fake-token" })),
  startNewGame: vi.fn((req, res) => res.status(200).json({ sessionId: "1" })),
  completeRoute: vi.fn((req, res) => res.status(200).json({ success: true })),
}));
```

Replaces the real handlers with simple stubs that immediately return success. This isolates the routing concern.

```
vi.mock("../src/middleware/middleware", () => ({
  authenticateJWT: (req: any, res: any, next: () => void) => {
```



```

    req.auth = { sessionId: 1, playerId: 1 };
    next();
  },
}));

```

Bypasses real JWT verification. The middleware stub injects `req.auth` directly and calls `next()`, simulating an authenticated request.

```

import app from "../../src/app";
import { Validate, startNewGame, completeRoute } from "../../src/api/api";

```

Imports the Express app (with mocks in place) and the handler stubs for assertion.

Test — "GET /api/health returns 200 with status ok"

```

const res = await request(app).get("/api/health");
expect(res.status).toBe(200);
expect(res.body.status).toBe("ok");
expect(res.body.timestamp).toBeDefined();

```

The health-check endpoint is implemented directly in the router (not mocked). Confirms it returns `200` with a `status` and `timestamp` field.

Test — "POST /api routes to Validate handler"

```

const res = await request(app).post("/api").send({ sessionId: 1 });
expect(res.status).toBe(200);
expect(vi.mocked(Validate)).toHaveBeenCalledTimes(1);

```

Sends a POST to `/api` and confirms the `Validate` handler stub was called exactly once.

Test — "POST /api/start-game routes to startNewGame handler" Same pattern for `/api/start-game` → `startNewGame`.

Test — "POST /api/complete-route routes to completeRoute handler" Same pattern for `/api/complete-route` → `completeRoute`.

Test — "protected routes require auth - returns 401 without token"

```

vi.doMock("../../src/middleware/middleware", () => ({
  authenticateJWT: (req: any, res: any) => res.status(401).json({ error: "Missing token" }),
}));
const { authenticateJWT } = await import("../../src/middleware/middleware");
const mockReq = { headers: {} } as any;
const mockRes = { status: vi.fn().mockReturnThis(), json: vi.fn() } as any;
const next = vi.fn();
authenticateJWT(mockReq, mockRes, next);
expect(mockRes.status).toHaveBeenCalledTimes(1);
expect(next).not.toHaveBeenCalled();

```

Uses `vi.doMock` (runtime mock, not hoisted) to load a version of the middleware that refuses the request. Calls it directly to confirm it returns `401` and does NOT call `next()` .

Frontend Tests

8. AccessibilityPage.test.tsx

Path: [frontend/testing/AccessibilityPage.test.tsx](#) **Purpose:** Tests the Accessibility settings page — High Contrast Mode checkbox, its persistence in `localStorage` , loading saved settings on mount, and the `FontSizeSlider` (value, range attributes, `localStorage` persistence, CSS custom property). TTS checkbox tests are currently commented out pending a fix.

```
import React from 'react';
import { render, screen, waitFor, fireEvent } from '@testing-library/react';
import { describe, it, expect, beforeEach, vi } from 'vitest';
import userEvent from '@testing-library/user-event';
import PageAccessibility from '../src/app/accessibility/page';
```

Imports React (required for JSX), Testing Library utilities (`fireEvent` used for the range slider), Vitest, `userEvent` for realistic click simulation, and the component under test.

```
beforeEach(() => {
  localStorage.clear();
});
```

Clears `localStorage` before each test so tests start with a clean state.

"should render Accessibility without crashing"

```
const { container } = render(<PageAccessibility />);
expect(container).toBeTruthy();
```

Basic smoke test — renders the component and confirms the DOM container exists.

"should render the High Contrast Mode checkbox"

```
const contrastCheckbox = screen.getByLabelText('High Contrast Mode');
expect(contrastCheckbox).toBeDefined();
```

Uses `getByLabelText` which queries for an input associated with a `<label>` containing "High Contrast Mode". Confirms the checkbox is rendered and accessible.

"should initialize High Contrast Mode checkbox as unchecked"

```
const contrastCheckbox = screen.getByLabelText('High Contrast Mode') as
HTMLInputElement;
expect(contrastCheckbox.checked).toBe(false);
```

Casts to `HTMLInputElement` to access `.checked`. Asserts the default state is `false`.

Note: Tests for "should render the Text-to-Speech checkbox", "should initialize Text-to-Speech checkbox as unchecked", "should toggle Text-to-Speech checkbox when clicked", "should not show TTSButton when TTS is unchecked", and "should show TTSButton when TTS is checked" are currently commented out.

"should toggle High Contrast Mode checkbox when clicked"

```
const user = userEvent.setup();
render(<PageAccessibility />);
const checkbox = screen.getByLabelText('High Contrast Mode') as HTMLInputElement;
expect(checkbox.checked).toBe(false);
await user.click(checkbox);
expect(checkbox.checked).toBe(true);
await user.click(checkbox);
expect(checkbox.checked).toBe(false);
```

`userEvent.setup()` creates a user-event instance that simulates realistic browser events. Clicks toggle the checkbox on and off, verifying both transitions.

"should render Back button"

```
const backButton = screen.getByText('Back');
expect(backButton).toBeDefined();
```

Confirms a `Back` navigation element exists on the page.

"should save settings to localStorage when toggled"

```
await user.click(checkbox);
await waitFor(() => {
  expect(localStorage.setItem).toHaveBeenCalledWith(
    'accessibilitySettings',
    expect.stringContaining('"highContrast":true')
  );
});
```

`waitFor` retries the assertion until it passes (accommodates async React state updates). Verifies the component persists settings to `localStorage` with the correct key and JSON content. Note:

`localStorage.setItem` must be a `vi.fn()` spy for `.toHaveBeenCalledWith()` to work — this is configured in the Vitest global setup file.

"should save multiple settings to localStorage from the accessibility menu"

```
await user.click(highContrastCheckbox);
await waitFor(() => {
  const lastCall = (localStorage.setItem as any).mock.calls.slice(-1)[0];
  const savedData = lastCall[1];
```

```
expect(savedData).toContain('"highContrast":true');
});
```

Clicks High Contrast and inspects the last `setItem` call. The TTS click and TTS assertion are currently commented out — only `highContrast` is verified.

"should load saved settings from localStorage on mount"

```
const mockSettings = { highContrast: true, TTS: true, buttonPressReadAloud: true,
autoReadAloud: true };
const getItemSpy = vi.spyOn(global.localStorage, 'getItem');
getItemSpy.mockReturnValue(JSON.stringify(mockSettings));
render(<PageAccessibility />);
await waitFor(() => {
  expect(highContrastCheckbox.checked).toBe(true);
});
expect(getItemSpy).toHaveBeenCalledWith('accessibilitySettings');
```

Pre-populates `localStorage` via a spy, renders the component, and confirms the `highContrast` checkbox reflects the saved value. The TTS checkbox assertion is commented out.

"should use nullish coalescing to default false when settings are undefined"

```
const mockSettings = { highContrast: undefined, TTS: undefined };
```

When stored settings have `undefined` values, the component uses `false` as default via `??`. Only the `highContrast` check is active; TTS assertion is commented out.

"should load only High Contrast Mode when saved as true" `highContrast: true`, `TTS: false` — confirms `highContrastCheckbox.checked === true`. TTS assertion commented out.

"should load only TTS when saved as true" `highContrast: false`, `TTS: true` — confirms `highContrastCheckbox.checked === false`. TTS assertion commented out (the test name is misleading given the TTS check is inactive).

"should use default values when localStorage is empty" With no stored settings, `highContrastCheckbox.checked === false`. TTS assertion commented out.

Font Size Slider Tests (all in the same `describe('AccessibilityPage Component')` block)

"should render the font size slider"

```
const slider = screen.getByLabelText(/Text Size/i) as HTMLInputElement;
expect(slider).toBeDefined();
```

Queries by the `aria-label` pattern — confirms the slider element is in the DOM.

"should initialize the font size slider at 100"

```
expect(slider.value).toBe('100');
```

Default state with no localStorage data — scale is 100%.

"should have correct min, max, and step attributes" Asserts `slider.min === '75'` , `slider.max === '150'` , `slider.step === '25'` — the four valid snap positions.

"should update slider value when changed"

```
fireEvent.change(slider, { target: { value: '125' } });
expect(slider.value).toBe('125');
```

Uses `fireEvent.change` (rather than `userEvent.click`) since range inputs don't toggle — they take a new value directly.

"should save fontSize to localStorage when slider changes" After a `fireEvent.change` to 125 , `waitFor` checks that `localStorage.setItem` was called with `'accessibilitySettings'` and JSON containing `"fontSize":125` .

"should load saved fontSize from localStorage on mount" Mocks `localStorage.getItem` to return `fontSize: 150` . After render, `waitFor` confirms `slider.value === '150'` .

"should default fontSize to 100 when not present in localStorage" Same mock setup but `fontSize` is intentionally omitted from the saved object. Confirms the `?? 100` default kicks in.

"should apply --font-scale CSS property on mount"

```
const fontScale = document.documentElement.style.getPropertyValue('--font-scale');
expect(fontScale).toBe('100%');
```

Confirms the `useEffect` at [L66–70](#) runs on mount and writes the CSS custom property.

"should update --font-scale CSS property when fontSize saved in localStorage is loaded" Mocks `fontSize: 125` in localStorage, confirms `--font-scale` is `'125%'` after mount.

"should include fontSize in the settings object saved to localStorage" Mocks `fontSize: 75` , confirms the next `setItem` call serializes `"fontSize":75` in the saved JSON.

9. HomePage.test.tsx

Path: [frontend/testing/HomePage.test.tsx](#) **Purpose:** Smoke-tests the Home page component to confirm key UI elements are rendered.

```
import { describe, it, expect, afterEach } from 'vitest';
import { render, screen, cleanup } from '@testing-library/react';
import PageHome from '../src/app/home/page';
```

Imports are minimal — no user interaction needed, just rendering and querying.

"should render without crashing" `container` is truthy after `render` .

"should render the main table structure"

```
const table = container.querySelector('.page_table');
expect(table).toBeTruthy();
```

Uses a CSS class selector to verify the page layout wrapper exists.

"should render Start New Pollinator Path button" `screen.getByText('Start New Pollinator Path')` — finds the button by its label text.

"should render Accessibility button" Same pattern for the Accessibility navigation button.

"should render the Pollinator Collection button" Same pattern for the Pollinator Collection button.

"should render Conner Prairie logo"

```
const logo = screen.getByAltText('Conner Prairie Logo');
expect(logo).toBeTruthy();
```

Finds the logo `<img>` by its `alt` attribute — important for accessibility correctness.

10. `Layout.test.tsx`

Path: [frontend/testing/Layout.test.tsx](#) **Purpose:** Tests the root layout wrapper (`RootLayout`) — the component that wraps every page in the Next.js app.

```
import RootLayout from '../src/app/layout';
```

"should render without crashing"

```
render(
  <RootLayout>
    <div>Test Child Content</div>
  </RootLayout>
);
expect(container).toBeTruthy();
```

Renders the layout with a single child and confirms no crash.

"should render children content"

```
const childContent = screen.getByText('Test Child Content');
expect(childContent).toBeDefined();
```

Confirms the layout actually renders its `children` prop.

"should render multiple children"

```
render(
  <RootLayout>
    <div>First Child</div>
```

```

    <div>Second Child</div>
  </RootLayout>
);
expect(screen.getByText('First Child')).toBeDefined();
expect(screen.getByText('Second Child')).toBeDefined();

```

Verifies the layout does not discard or hide any of its children.

11. Middleware.test.tsx

Path: [frontend/testing/Middleware.test.tsx](#) **Purpose:** Tests the Next.js middleware that enforces route access — allowing public paths and redirecting unknown ones.

```

import { middleware } from '../src/middleware';

function makeReq(path: string, url = `http://localhost${path}`) {
  return { nextUrl: { pathname: path }, url } as any;
}

```

`makeReq` is a helper that constructs a minimal fake `NextRequest` object with only the fields the middleware reads (`nextUrl.pathname` and `url`).

"allows public paths"

```

const res = middleware(makeReq('/'));
expect(res.status).toBe(200);
expect(res.headers.get('location')).toBeNull();

```

A `200` response with no `location` header means the middleware passed the request through (no redirect).

"allows next/_next, api and static files"

```

expect(middleware(makeReq('/_next/static/file.js')).status).toBe(200);
expect(middleware(makeReq('/api/validate')).status).toBe(200);
expect(middleware(makeReq('/favicon.ico')).status).toBe(200);

```

Checks that internal Next.js paths, API routes, and static assets are allowed through without redirect.

"allows listed PUBLIC_PATHS like /route and /pollinator-collection" Specific application routes that should always be accessible.

"redirects unknown paths to /redirecting"

```

const res = middleware(makeReq('/not-a-real-path', 'http://localhost/not-a-real-path'));
expect(res.status).toBe(307);
expect(res.headers.get('location')).toBe('http://localhost/redirecting');

```

307 Temporary Redirect to the /redirecting page. The location header is an absolute URL built from the incoming request's url field.

12. PollinatorCollectionPage.test.tsx

Path: [frontend/testing/PollinatorCollectionPage.test.tsx](#) **Purpose:** Tests the Pollinator Collection page, which displays pollinators the player has discovered. Uses a mocked routeService to control what completed routes exist. **Contributors:** Campbell (bsucs) — wrote "updates when new pollinators are added" and "ignores invalid numeric ids left in storage" ; also refactored mocks to use vi.mocked and updated tests to match new backend API entry point.

```
vi.mock('@app/services/routeService', () => ({
  getCompletedRoutes: vi.fn(() => []),
  setCompletedRoutes: vi.fn(),
  addCompletedRoute: vi.fn(),
}));
```

Mocks the route service so tests control what pollinators are "completed" without real localStorage.

```
beforeEach(() => {
  vi.mocked(getCompletedRoutes).mockReturnValue([]);
});
afterEach(() => {
  cleanup();
  vi.clearAllMocks();
});
```

Each test starts with an empty collection. cleanup() unmounts the rendered component after each test.

"should render without crashing" — Basic smoke test.

"should render the Conner Prairie logo" — Finds .

"should render the main table structure" — Finds .page_table wrapper.

"should render a Home button for navigation"

```
const homeButton = await screen.findByRole('link', { name: /home/i });
expect(homeButton).toHaveAttribute('href', '/home');
expect(homeButton).toHaveClass('home_button');
await user.click(homeButton);
await waitFor(() => expect(homeButton).toHaveFocus());
```

findByRole (async) waits for the element. Checks both href and CSS class. Clicking the link and confirming focus verifies keyboard accessibility.

"should render a Settings button for navigation"

```
const settingsButton = screen.getByRole('link', { name: /\@/i });
expect(settingsButton).toHaveAttribute('href', '/accessibility');
```



```
expect(settingsButton).toHaveClass('settings-button');
```

The gear icon link navigates to the accessibility settings page.

"updates when new pollinators are added"

```
expect(screen.getByText('Bee')).toBeNull();
window.dispatchEvent(new CustomEvent('completedRoutesUpdated', { detail: ['Bee']
})));
await waitFor(() => expect(screen.getByText('Bee')).toBeTruthy());
window.dispatchEvent(new CustomEvent('completedRoutesUpdated', { detail: ['Bee',
'Bat'] })));
await waitFor(() => expect(screen.getByText('Bat')).toBeTruthy());
```

The component listens to `completedRoutesUpdated` custom events. Dispatching them simulates the route service notifying the page of new discoveries. Both pollinators appear incrementally.

"ignores invalid numeric ids left in storage"

```
vi.mocked(getCompletedRoutes).mockReturnValue(['Bee', '5', 'Bat']);
render(<PagePollinatorCollection />);
await waitFor(() => {
  expect(screen.getByText('Bee')).toBeTruthy();
  expect(screen.getByText('Bat')).toBeTruthy();
});
expect(screen.getByText('5')).toBeNull();
expect(vi.mocked(setCompletedRoutes)).toHaveBeenCalledWith(['Bee', 'Bat']);
```

Numeric-only strings (`'5'`) are invalid pollinator names. The component filters them out and calls `setCompletedRoutes` with the cleaned list.

13. RedirectPage.test.tsx

Path: [frontend/testing/RedirectPage.test.tsx](#) **Purpose:** Tests the `RedirectingPage` (shown when a user hits an invalid route) and the shared `RouteText` component.

```
let replaceMock = vi.fn();
vi.mock('next/navigation', () => ({
  useRouter: () => ({ replace: replaceMock }),
})));
```

Mocks Next.js's `useRouter`. The page uses `router.replace()` to programmatically navigate after a countdown. The mock intercepts this call.

```
beforeEach(() => {
  replaceMock = vi.fn();
  vi.useFakeTimers();
});
afterEach(() => {
  vi.restoreAllMocks();
});
```

```
vi.useRealTimers();
});
```

`vi.useFakeTimers()` takes control of `setTimeout` / `setInterval` to prevent the page's real countdown timer from executing during tests (which would trigger navigation and cause side effects). None of these tests actually advance the fake clock — the fake timers are used purely to freeze time, not to simulate it passing. Restored after each test via `vi.useRealTimers()`.

"should render without crashing" — Basic smoke test.

"should render the main table structure" — `.page_table` wrapper exists.

"should render the Conner Prairie logo" — Logo image present.

"should display Enter your Session ID text"

```
const text = screen.getByText('Redirecting to the activity homepage in 3');
expect(text).toBeDefined();
```

The page shows a countdown message starting at 3. Note: this test name is a copy-paste artifact from `RootPage.test.tsx` — the `RedirectingPage` shows a countdown, not a session ID prompt. The assertion itself is correct.

"RouteText displays with labelText"

```
render(<RouteText labelText="Custom heading" />);
expect(screen.getByRole('heading', { name: /custom heading/i })).toBeDefined();
```

Tests the `RouteText` component in isolation — confirms it renders its `labelText` prop as a heading element.

14. `RootPage.test.tsx`

Path: [frontend/testing/RootPage.test.tsx](#) **Purpose:** Tests the root page (/) — the session ID entry screen where users start the experience.

```
import PageSessionId from '../src/app/page';
```

"should render without crashing" — Basic smoke test.

"should render the main table structure" — `.page_table` wrapper.

"should render the Conner Prairie logo" — Logo present.

"should display Enter your Session ID text"

```
const text = screen.getByRole('heading');
expect(text).toBeDefined();
```

Queries for any heading element — confirms the page has a title/heading.

"should render a Validate button"

```
const button = screen.getByText('Validate');
expect(button).toBeDefined();
```

The primary CTA button for submitting the session ID.

"should have a sessionId textbox"

```
const textbox = container.querySelector('.sessionId-Textbox') as HTMLInputElement | null;
expect(textbox).toBeTruthy();
```

Finds the text input by its CSS class — confirms the input field is rendered.

15. RootPageAPI.test.tsx

Path: [frontend/testing/RootPageAPI.test.tsx](#) **Purpose:** Tests the root page's API interaction behavior — what happens when the user types a session ID and clicks Validate, covering success and failure scenarios.

```
beforeEach(() => {
  setItemMock = vi.fn();
  vi.spyOn(Storage.prototype, 'setItem').mockImplementation(setItemMock as any);
  vi.spyOn(Storage.prototype, 'getItem').mockImplementation(vi.fn() as any);
  vi.spyOn(Storage.prototype, 'removeItem').mockImplementation(vi.fn() as any);
});
```

Spies on all `localStorage` methods to intercept reads and writes without a real browser storage.

"validates session, stores JWT and sessionId, and shows success UI"

```
const jsonMock = vi.fn().mockResolvedValue({ token: 'jwt-token' });
vi.spyOn(global, 'fetch').mockResolvedValueOnce({
  ok: true,
  json: jsonMock,
} as any);
render(<PageSessionId />);
```

Mocks `fetch` to return a successful response with a JWT token. The test renders the component but stops short of exercising full flow — primarily ensures no crash when `fetch` is available and succeeds.

"handles network error and shows generic error message"

```
vi.spyOn(global, 'fetch').mockRejectedValueOnce(new Error('network down'));
render(<PageSessionId />);
const input = screen.getByPlaceholderText('Enter your session code') as HTMLInputElement;
await user.type(input, '12345');
const button = screen.getByRole('button', { name: /validate/i });
await user.click(button);
```

```

await waitFor(() => {
  expect(screen.getByText('Failed to validate session. Please try
again.')).toBeInTheDocument();
});
expect(setItemMock).not.toHaveBeenCalled();

```

Simulates a network failure. Types a session code, clicks Validate, and asserts:

1. An error message appears in the UI.
2. Nothing was written to `localStorage` (JWT not stored on failure).

16. RoutePageFlow.test.tsx

Path: [frontend/testing/RoutePageFlow.test.tsx](#) **Purpose:** End-to-end UI flow tests for the main game Route page. Simulates a player navigating through route nodes, fact nodes, and completing routes. **Contributors:** Gregg Reed — added the dedicated `/api/complete-route` POST handler inside `createFetchMock` to prevent the generic `/api` POST mock from intercepting it (bug fix). Campbell (bsucs) — updated the fetch mock to align with the refactored backend API entry point.

```

const createFetchMock = (routeData: any) => {
  return vi.fn((input: any, opts?: any) => {
    const url = typeof input === 'string' ? input : (input?.url ?? '');
    const method = String(opts?.method || 'GET').toUpperCase();

    if (url.includes('/api/complete-route') && method === 'POST') {
      return Promise.resolve({ ok: true, text: async () => JSON.stringify({
        success: true,
        routesCompleted: 1,
        newActiveRoute: routeData.length > 1 ? routeData[1] : null,
      })});
    }
    if (url.includes('/api/start-game') && method === 'POST') {
      return Promise.resolve({ ok: true, text: async () => JSON.stringify({
        player: { activeRoute: routeData[0] ?? null }
      })});
    }
    return Promise.resolve({ ok: false, text: async () => 'Not found' } as any);
  });
};

```

A factory that creates a smart `fetch` mock. Routes `/api/start-game` to return the first route, `/api/complete-route` to advance to the second route (simulating the backend's game logic). Any other URL returns a `404`.

```

beforeEach(() => {
  storageMap = new Map<string, string>();
  storageMap.set('authToken', 'fake-token');
  Object.defineProperty(window, 'localStorage', { value: { ... }, writable: true });
});
afterEach(() => {
  vi.restoreAllMocks();
});

```

```
delete (global as any).fetch;
storageMap.clear();
});
```

Sets up an in-memory `localStorage` substitute with a pre-seeded auth token. Cleans up after each test.

"fetches and displays route data on mount"

```
global.fetch = createFetchMock(routeData);
render(<RoutePage />);
await waitFor(() => expect(screen.getByText('Node 1')).toBeInTheDocument());
```

The page auto-fetches the route on mount. Waits for the first node text to appear.

"Home button has correct link"

```
const homeLink = screen.getByRole('link', { name: /home/i });
expect(homeLink).toHaveAttribute('href', '/home');
```

Confirms the navigation link is present with the correct destination.

"displays fact node after popup dismissed"

```
await waitFor(() => expect(screen.getByText(/You discovered/)).toBeInTheDocument());
await user.click(screen.getByRole('button', { name: /continue/i }));
await user.click(screen.getByRole('button', { name: /next/i }));
await waitFor(() => expect(screen.getByText('Fact 1')).toBeInTheDocument());
```

The page shows a "You discovered" popup on the first load. After dismissing with "Continue" then clicking "Next", the fact node (`factNodes`) is displayed.

"navigates to next route node when Next clicked"

```
await user.click(screen.getByRole('button', { name: /next/i }));
await waitFor(() => expect(screen.getByText('Node 2')).toBeInTheDocument());
```

Confirms forward navigation through `routeNodes` .

"navigates to previous route node when Previous clicked"

```
await user.click(screen.getByRole('button', { name: /next/i }));
await waitFor(() => expect(screen.getByText('Node 2')).toBeInTheDocument());
await user.click(screen.getByRole('button', { name: /previous/i }));
await waitFor(() => expect(screen.getByText('Node 1')).toBeInTheDocument());
```

Forward then backward — confirms bidirectional navigation.

"displays route header when not on start node"

```
await user.click(screen.getByRole('button', { name: /next/i }));
await waitFor(() => expect(screen.getByText('Route Node 1')).toBeInTheDocument());
const routeHeader = document.querySelector('.route_header');
expect(routeHeader).toBeInTheDocument();
```

The route header (showing route name/progress) only appears after leaving the starting node.

"displays 'Did you know?' header on fact nodes"

```
await waitFor(() => expect(screen.getByText('Fact about
bees')).toBeInTheDocument());
expect(screen.getByText(/Did you know/i)).toBeInTheDocument();
```

When on a fact node, the header changes to "Did you know?" to visually distinguish facts from route content.

"Start New Route button restarts the flow with a new random route"

```
vi.spyOn(Math, 'random').mockReturnValue(0);
```

Mocks `Math.random` to control which route is selected. Walks the full route (nodes → discovery popup → facts → end of route → survey). Clicks "Start New Route" and confirms the second route's content appears.

"displays survey popup at the end of a route"

```
await waitFor(() => expect(screen.getByText(/Optional
Survey/i)).toBeInTheDocument());
```

After exhausting all fact nodes, an optional survey popup is shown.

17. `jwtService.test.tsx`

Path: [frontend/testing/jwtService.test.tsx](#) **Purpose:** Unit-tests the JWT service functions: local storage management (`saveJwt` / `getJwt` / `clearJwt`) and API-based JWT fetching/authenticated requests.

```
import { saveJwt, getJwt, clearJwt, fetchJwtFromApi, apiFetchAuthenticated } from
'../src/app/services/jwtService';
```

The `beforeEach` / `afterEach` pattern mirrors other tests — in-memory `storageMap` replaces real `localStorage` .

`saveJwt` / `getJwt` / `clearJwt`

"saves and retrieves JWT from localStorage"

```
saveJwt('test-token-123');
expect(getJwt()).toBe('test-token-123');
```

Round-trip: save a token, read it back.

"returns null when no JWT exists"

```
expect(getJwt()).toBeNull();
```

Empty storage returns `null` .

"clears JWT from localStorage"

```
saveJwt('test-token-123');
clearJwt();
expect(getJwt()).toBeNull();
```

After clearing, `getJwt` returns `null` again.

fetchJwtFromApi

"fetches JWT and saves to localStorage"

```
global.fetch = vi.fn(() => Promise.resolve({
  ok: true,
  text: async () => JSON.stringify({ token: 'fetched-token' })
} as any));
const token = await fetchJwtFromApi(12345);
expect(token).toBe('fetched-token');
expect(getJwt()).toBe('fetched-token');
```

Mocks `fetch` to return a valid JWT response. Confirms the function returns the token AND stores it in `localStorage` .

"throws error on failed handshake"

```
global.fetch = vi.fn(() => Promise.resolve({
  ok: false, status: 401, ...
} as any));
await expect(fetchJwtFromApi(99999)).rejects.toThrow('JWT handshake failed');
```

A non-OK HTTP response should throw a descriptive error.

"throws error when response has no token"

```
text: async () => JSON.stringify({ noToken: true })
await expect(fetchJwtFromApi(12345)).rejects.toThrow('JWT handshake returned no token');
```

The response parses as JSON but lacks the `token` field — should throw.

"throws error on invalid JSON response"

```
text: async () => 'not valid json'
```

```
await expect(fetchJwtFromApi(12345)).rejects.toThrow('JWT parse error');
```

Malformed response body triggers a parse error.

apiFetchAuthenticated

"makes authenticated request with JWT header"

```
saveJwt('my-jwt-token');
global.fetch = vi.fn(() => Promise.resolve({ ok: true, text: async () =>
JSON.stringify({ data: 'test' }) } as any));
const result = await apiFetchAuthenticated<{ data: string }>('/api/test');
expect(result).toEqual({ data: 'test' });
expect(global.fetch).toHaveBeenCalled();
```

With a JWT in storage, the function calls `fetch` and returns the parsed JSON body. The test confirms `fetch` was invoked and the response is correctly deserialized — it does not directly assert that the `Authorization` header was sent in the request.

"throws error when no JWT exists"

```
await expect(apiFetchAuthenticated('/api/test')).rejects.toThrow('Missing JWT');
```

Without a stored token, the function refuses to make the request.

"throws error on failed API response"

```
global.fetch = vi.fn(() => Promise.resolve({ ok: false, status: 500, ... } as any));
await expect(apiFetchAuthenticated('/api/test')).rejects.toThrow('API error');
```

A server error response throws an `API error`.

"throws error on invalid JSON response"

```
text: async () => 'not valid json'
await expect(apiFetchAuthenticated('/api/test')).rejects.toThrow('JSON parse error');
```

An unparseable response body throws a `JSON parse error`.

18. routeService.test.tsx

Path: [frontend/testing/routeService.test.tsx](#) **Purpose:** Unit-tests the route service — localStorage-backed completed route management, random route selection, and route filtering. **Contributors:** Campbell (bsucs) — wrote "dispatches completedRoutesUpdated event on successful add" and "adds multiple different routes".

The `beforeEach / afterEach` setup is the same in-memory `localStorage` pattern plus a mock for `document.cookie`.

`getCompletedRoutes / setCompletedRoutes`

"returns empty array when no completed routes exist"

```
expect(getCompletedRoutes()).toEqual([]);
```

Empty `localStorage` → empty array.

"saves and retrieves completed routes"

```
setCompletedRoutes(['route1', 'route2']);  
expect(getCompletedRoutes()).toEqual(['route1', 'route2']);
```

Round-trip serialization: array is JSON-stringified on save, parsed on retrieval.

"handles empty array"

```
setCompletedRoutes([]);  
expect(getCompletedRoutes()).toEqual([]);
```

Edge case: saving an empty array is valid and retrieves correctly.

addCompletedRoute

"adds a new route to completed routes"

```
addCompletedRoute('route1');  
expect(getCompletedRoutes()).toContain('route1');
```

After adding, the route ID is in the list.

"does not add duplicate routes"

```
addCompletedRoute('route1');  
addCompletedRoute('route1');  
expect(getCompletedRoutes().filter(r => r === 'route1')).toHaveLength(1);
```

Adding the same route twice does not create duplicates.

"does not add empty routeId"

```
addCompletedRoute('');  
expect(getCompletedRoutes()).toEqual([]);
```

Empty string is not a valid route ID — ignored.

"adds multiple different routes"

```
addCompletedRoute('route1');  
addCompletedRoute('route2');
```

```
addCompletedRoute('route3');
expect(getCompletedRoutes()).toEqual(['route1', 'route2', 'route3']);
```

Confirms insertion order is preserved.

"dispatches completedRoutesUpdated event on successful add"

```
const dispatchSpy = vi.spyOn(window, 'dispatchEvent');
addCompletedRoute('routeA');
expect(dispatchSpy).toHaveBeenCalled();
const calledWith = dispatchSpy.mock.calls[0][0] as CustomEvent;
expect(calledWith.type).toBe('completedRoutesUpdated');
expect(calledWith.detail).toEqual(getCompletedRoutes());
```

After a successful add, a `CustomEvent` is dispatched on `window` so the Pollinator Collection page can react. Asserts both the event type and that its `detail` contains the updated routes array.

getRandomInt

"returns integer within range"

```
for (let i = 0; i < 100; i++) {
  const result = getRandomInt(0, 10);
  expect(result).toBeGreaterThanOrEqual(0);
  expect(result).toBeLessThan(10);
}
```

Runs 100 iterations to statistically confirm the random integer always falls within `[min, max)`.

"handles min equal to max minus one"

```
const result = getRandomInt(5, 6);
expect(result).toBe(5);
```

When the range is a single integer, only that value can be returned.

"handles negative range"

```
for (let i = 0; i < 50; i++) {
  const result = getRandomInt(-10, 0);
  expect(result).toBeGreaterThanOrEqual(-10);
  expect(result).toBeLessThan(0);
}
```

Negative ranges work correctly.

getRandomRoute

"returns a route from available routes"

```
const routes: Route[] = [
  { routeId: '1', routeNodes: [], factNodes: [] },
  { routeId: '2', routeNodes: [], factNodes: [] }
];
const result = getRandomRoute(routes);
expect(routes).toContain(result);
```

The returned route is one of the provided options.

"returns the only route when single route provided"

```
const result = getRandomRoute(routes);
expect(result).toEqual(routes[0]);
```

With only one route, it must always be returned.

filterAvailableRoutes

"filters out completed routes"

```
const routes = [{ routeId: '1' }, { routeId: '2' }, { routeId: '3' }];
const result = filterAvailableRoutes(routes, ['1', '3']);
expect(result).toHaveLength(1);
expect(result[0].routeId).toBe('2');
```

Removes routes whose IDs are in the completed list. Only route `'2'` remains.

"returns all routes when none are completed"

```
const result = filterAvailableRoutes(routes, []);
expect(result).toHaveLength(2);
```

Empty completed list → all routes are available.

"returns empty array when all routes are completed"

```
const result = filterAvailableRoutes(routes, ['1', '2']);
expect(result).toHaveLength(0);
```

All routes completed → no routes available.

Portal Tests

19. surveyPortalStructure.test.tsx

Path: [portal/testing/surveyPortalStructure.test.tsx](#) **Purpose:** Structural render tests for the `PortalPage` component. Confirms that all key UI elements are present after the redesign — no user interaction required. 14 tests total.

Setup: Renders `<PortalPage />` for each test. No mocks needed.

"should render without crashing" — Container is truthy.

"should render the main table structure" — `.page_table` wrapper exists.

"should render the Conner Prairie logo" — `<img alt="Conner Prairie Logo">` present.

"should render 'Search' button" — `screen.getByText('Search')` is truthy.

"should render session ID input" — `data-testid="sessionIdInput"` element found.

"should render '.results_table'" — Results table container present.

"should render 'Pollinator Habitat Admin Portal' header" — Portal header text visible.

"should render 'Search by Date or Date Range' section header" — `SearchHeader` for date section visible.

"should render 'Search by Session ID Number' section header" — `SearchHeader` for session ID section visible.

"should render Date Range checkbox" — Checkbox for enabling date range mode present.

"should render Start Date input" — `data-testid="startDateInput"` element found.

"should render End Date input" — `data-testid="endDateInput"` element found.

"should render 'Start Date' and 'End Date' labels" — Both label texts visible.

"should render 'Clear search' button" — Reset button text visible.

"should render 'Clear table' button" — Table reset button text visible.

20. `surveyPortalFunction.test.tsx`

Path: [portal/testing/surveyPortalFunction.test.tsx](#) **Purpose:** Functional tests for the redesigned portal. Two describe blocks covering input enforcement, search/reset behavior, and fetch integration.

Setup: Renders `<PortalPage />` with `userEvent.setup()`. Fetch is mocked where needed.

describe('Portal Session ID Input') — 7 tests

"enforces 9 character limit on session ID" Types 11 chars → asserts input value is capped at 9 digits via `sanitizeSessionIdInput`.

"displays placeholder in table if no results are found" `SearchResultsTable` with empty array shows "No results" and "--/--/---".

"End date input is disabled if date range is not enabled" End date `DateInput` is disabled by default (checkbox unchecked).

"End date input is enabled if date range is enabled" Click the Date Range checkbox → end date input becomes enabled.

"clear search removes all search terms and resets date range" Fills all search fields and checks the range checkbox, then clicks 'Clear search' → all fields blank and checkbox unchecked.

"clear table removes all search results" Mocks fetch to return data, clicks Search, sees results, then clicks 'Clear table' → table resets to "No results".

"error message is displayed if search criteria is invalid" Types a partial (invalid) session ID, clicks Search → alert or error message shows "Invalid search criteria".

describe('Search by Session ID') — 8 tests covering fetchChildrenAdultsBySessionId

1. Returns one aggregated row summing adults and children, with `sessionDate` .
 2. Returns row with date `--` when API omits `sessionDate` .
 3. Returns empty array when API `data` is empty.
 4. Throws when response is not JSON (returns `text/html`).
 5. Throws when API returns `success: false` .
 6. Full UI test: search by session ID displays one row with Session ID, Date, Adults, Children, Total columns.
 7. Full UI test: shows "No results" when API returns empty data.
 8. Full UI test: shows error alert when API fails.
-

21. inputValidator.test.tsx (NEW)

Path: [portal/testing/inputValidator.test.tsx](#) **Purpose:** Unit tests for `isValidDateInput` and `isValidSessionIdInput` in `portal/src/services/inputValidator.ts` .

describe('isValidDateInput') — 4 tests

"should return true for a valid date" — `'2026-01-01'` → `true`

"should return false for an invalid date" — `'2026-01-01-01'` → `false`

"should return false for invalid session ID" — 8-digit string → `isValidSessionIdInput` returns `false`

"should accept valid 9 digit session ID" — 9-digit string → `isValidSessionIdInput` returns `true`

22. layout.test.tsx (NEW)

Path: [portal/testing/layout.test.tsx](#) **Purpose:** Tests the portal `RootLayout` component.

describe('RootLayout Component') — 3 tests

"should render without crashing" — Container is truthy.

"should render children content" — Child text `"Test Child Content"` visible after render.

"should render multiple children" — Two children both visible after render.

23. exportTool.test.tsx

Path: [portal/testing/exportTool.test.tsx](#) **Purpose:** Structural render tests for the `ExportPage` component (`portal/src/app/export/page.tsx`). Confirms all key UI elements are present. 9 tests total.

Setup: Renders `<ExportPage />` for each test. No mocks needed.

"should render without crashing" — Container is truthy.

"should render the main table structure" — `.page_table` wrapper exists.

"should render the Conner Prairie logo" — `<img alt="Conner Prairie Logo">` present.

"should render the Home button" — `data-testid="HomeButton"` element found.

"should render the Quick Export Tool header" — `screen.getByText('Quick Export Tool')` is truthy.

"should render the reporting period checkboxes" — Six checkboxes present:
`Export1MonthCheckbox` , `Export3MonthsCheckbox` , `Export6MonthsCheckbox` ,
`Export1YearCheckbox` , `Export3YearsCheckbox` , `ExportAllTimeCheckbox` .

"should render the data option checkboxes" — Two checkboxes present:
`ExportSurveyDataCheckbox` , `ExportRouteDataCheckbox` .

"should render the reporting period header" — `'Choose a reporting period:'` text visible.

"should render the data option header" — `'Choose a data report option:'` text visible.

24. `portalMenu.test.tsx`

Path: [portal/testing/portalMenu.test.tsx](#) **Purpose:** Structural render tests for the `MenuPage` component (`portal/src/app/menu/page.tsx`). 6 tests total.

Setup: Renders `<MenuPage />` for each test. No mocks needed.

"should render without crashing" — Container is truthy.

"should render the main table structure" — `.page_table` wrapper exists.

"should render the Conner Prairie logo" — `<img alt="Conner Prairie Logo">` present.

"should render the Survey Data button" — `data-testid="SurveyDataButton"` found.

"should render the Route Data button" — `data-testid="RouteDataButton"` found.

"should render the Quick Export Tool (Coming Soon) button" — `data-testid="QuickExportToolButton"` found.

25. `splashPage.test.tsx`

Path: [portal/testing/splashPage.test.tsx](#) **Purpose:** Structural render tests for the root splash/auth page (`portal/src/app/page.tsx`). 6 tests total.

Setup: Renders root `<MenuPage />` (the default export of `page.tsx`) for each test. No mocks needed.

"should render without crashing" — Container is truthy.

"should render the main table structure" — `.page_table` wrapper exists.

"should render the Conner Prairie logo" — `<img alt="Conner Prairie Logo">` present.

"should render the UNAUTHORIZED USE IS PROHIBITED header" — Header text visible.

"should render challenge text header" — "By clicking 'To Menu' you confirm you are an authorized administrator for this system." visible.

"should render the To Menu button" — data-testid="ToPortalMenuButton" found.

New Backend Tests

8. databaseOperations/getChildrenAndAdultBySession.test.ts

Path: [backend/tests/databaseOperations/getChildrenAndAdultBySession.test.ts](#) **Purpose:** Tests `getChildrenAndAdultsBySession.getChildrenAndAdultsBySessionId(sessionId)` using a Prisma mock. 6 tests.

Mock: `prisma.playerSession.findMany`

Test 1 — "returns numChildren, numAdults, and playerId for a given sessionId" Mocks `findMany` returning 2 results. Verifies array length is 2.

Test 2 — "returns an empty array when no player sessions exist for the given sessionId" Mocks `findMany` returning `[]`. Result is empty array.

Test 3 — "queries with the correct sessionId" Verifies `findMany` was called with `{ where: { sessionId: 42 }, select: { playerId: true, numChildren: true, numAdults: true } }`.

Test 4 — "returns correct shape for a single player session" Checks that result contains `playerId`, `numChildren`, and `numAdults` properties.

Test 5 — "handles a session with multiple players correctly" Mocks 3 players — checks array length is 3.

Test 6 — "propagates errors thrown by prisma" Mocks `findMany` to reject. Expects the error to propagate (throws).

9. databaseOperations/getChildNumAndAdultDateToForever.test.ts

Path: [backend/tests/databaseOperations/getChildNumAndAdultDateToForever.test.ts](#) **Purpose:** Tests `getChildNumAndAdultDateToForever.getChildNumAndAdultDataToForever(startDate)` using a Prisma mock. 5 tests.

Mock: `prisma.playerSession.findMany`

Test 1 — "returns results when records exist on or after startDate" Mocks `findMany` returning 2 results with date fields. Verifies both are returned.

Test 2 — "returns an empty array when no records exist on or after startDate" A far-future date returns an empty array.

Test 3 — "queries with the correct gte filter on createdAt" Verifies `findMany` was called with:

```
{ where: { createdAt: { gte: startDate } }, select: { createdAt, updatedAt, playerId, sessionId, numAdults, numChildren } }
```

Test 4 — "returns only the selected fields" Verifies result objects have all 6 selected fields.

Test 5 — "propagates errors thrown by prisma" Expects throw on rejection.

10. databaseOperations/getActiveRouteForPlayerSession.test.ts

Path: [backend/tests/databaseOperations/getActiveRouteForPlayerSession.test.ts](#) **Purpose:** Tests `getActiveRouteForPlayerSessionService.getActiveRouteForPlayerSession(playerSessionId)` . 7 tests.

Mocks: `prisma.routeCycle.findFirst` and `LoadRouteDTOService.loadRouteDTO`

Test 1 — "returns a RouteDTO when an active route cycle exists" Mocks `findFirst` returning `{ routeId: 99 }` , `loadRouteDTO` returning a mock DTO. Verifies full call arguments.

Test 2 — "returns null when no active route cycle exists" `findFirst` returns `null` → result is `null` .

Test 3 — "returns null when active cycle has a null routeId" `findFirst` returns `{ routeId: null }` → result is `null` .

Test 4 — "queries with the correct playerSessionId" Verifies `findFirst` called with `where.playerSessionId: 42` .

Test 5 — "returns the most recent active cycle when multiple exist (orderBy desc)" Verifies `orderBy: { id: 'desc' }` in the query.

Test 6 — "propagates errors thrown by prisma" `findFirst` rejects → expects throw.

Test 7 — "propagates errors thrown by LoadRouteDTOService" `loadRouteDTO` rejects → expects throw.

11. databaseOperations/hasSession.test.ts

Path: [backend/tests/databaseOperations/hasSession.test.ts](#) **Purpose:** Tests `HasSessionService.hasSession(sessionId)` with a Prisma mock and fake timers. 8 tests.

Setup: Uses `vi.useFakeTimers()` / `vi.setSystemTime()` for date-based tests. Helper `localDate(dayOffset, hours)` creates dates relative to today in local time.

Test 1 — "queries with the correct sessionId" Verifies `findUnique` called with `{ where: { id: 42 }, select: { id: true, sessionDate: true } }` .

Test 2 — "returns false when session is not found" `findUnique` returns `null` → `false` .

Test 3 — "returns true when session date is today" Session date is local today at 10am → `true` .

Test 4 — "returns false when session date is in the future" Session date is tomorrow → `false` .

Test 5 — "returns false when session date is in the past" Session date is yesterday → `false` .

Test 6 — "returns true for a session at the very start of today (midnight)" Session at exactly local midnight → `true` .

Test 7 — "returns false for a session at exactly midnight of the next day" Session at midnight of the next calendar day → `false` .

Test 8 — "propagates errors thrown by prisma" `findUnique` rejects → expects throw.

12. databaseOperations/getOrCreatePlayerSession.test.ts

Path: [backend/tests/databaseOperations/getOrCreatePlayerSession.test.ts](#) **Purpose:** Tests `getOrCreatePlayerSessionService.getOrCreatePlayerSession(sessionId, playerId)` . 4 tests.

Mocks: `prisma.playerSession.createMany` , `prisma.playerSession.findUniqueOrThrow` , `checkSessionService.requireSession`

Test 1 — "creates and returns a new player session when none exists" `createMany` returns `{ count: 1 }` , `findUniqueOrThrow` returns `{ id: 42, playerId: 7 }` . Result: `{ playerIdSessionId: 42, playerId: 7 }` .

Test 2 — "returns existing session without creating a duplicate" `createMany` returns `{ count: 0 }` (already exists). `findUniqueOrThrow` still returns the row. Same result shape.

Test 3 — "throws when the session does not exist" `requireSession` rejects with "Invalid session ID" → error propagates.

Test 4 — "throws when playerId is missing" `undefined` `playerId` → throws "Missing playerId" .

New Frontend Tests

24. surveyStorageService.test.tsx

Path: [frontend/testing/surveyStorageService.test.tsx](#) **Purpose:** Tests `getSurveyResponse()` and `setSurveyResponse()` in `services/surveyStorageService` . Uses manual `localStorage` mock via `Object.defineProperty` . 4 tests.

`describe('surveyStorageService')`

"returns null when nothing is stored" Empty `localStorage` → `getSurveyResponse()` returns `null` .

"saves and retrieves survey response with children and adults" `setSurveyResponse(2, 3, 5)` → `getSurveyResponse()` returns `{ children: 2, adults: 3, totalPartySize: 5 }` .

"stores submittedAt when saving" Raw `localStorage` value after `setSurveyResponse` contains `totalPartySize` and a `submittedAt` ISO string.

"returns null for invalid stored data" Corrupt JSON in `localStorage` → `getSurveyResponse()` returns `null` .

25. UAT/OneTimePartyASizeSurvey.test.tsx

Path: [frontend/testing/UAT/OneTimePartyASizeSurvey.test.tsx](#) **Purpose:** End-to-end UAT test simulating a full two-route play session to verify the party size survey appears exactly once.

Flow: Renders `PageHome` → `RoutePage` (bat route) → walks through all stops and facts → survey (`PartySizeSurveyPopUp`) appears → Submit → `StartNewRouteButton` → advances to next route (human route) → walks to the final fact card → survey does **not** appear again.

Key assertions:

- `PartySizeSurveyPopUp` appears on first route completion (at the final fact card).
 - After Submit, "Start New Route" button becomes available.
 - Survey does NOT appear again on the second route's final fact card.
 - `newActiveRoute` from `/api/complete-route` correctly advances to route 2.
-

26. `UAT/PollinatorCollection.test.tsx`

Path: [frontend/testing/UAT/PollinatorCollection.test.tsx](#) **Purpose:** End-to-end test: play a route to the pollinator reveal, go home, navigate to Pollinator Collection.

Key assertions:

- "Discovered Pollinators" heading is visible on the collection page.
 - The `bat` pollinator (by `data-testid`) appears in the collection after completing the bat route.
-

27. `UAT/RepeatableRoutePlay.test.tsx`

Path: [frontend/testing/UAT/RepeatableRoutePlay.test.tsx](#) **Purpose:** End-to-end test verifying route repeatability across multiple completions. Plays the bat route all the way through (all stops + all facts), clicks "Start New Route", then verifies the first stop of the next route (human) appears.

Key assertions:

- After completing one full route and clicking "Start New Route", the next route's first stop is displayed.
 - Confirms the app does not get stuck after a route completion.
-

New Portal Tests

30. `inputValidator.test.tsx` (NEW)

Path: [portal/testing/inputValidator.test.tsx](#) See [section 21](#) above.

31. `layout.test.tsx` (NEW)

Path: [portal/testing/layout.test.tsx](#) See [section 22](#) above.

Backend Unit Tests — Campbell (`unit/campbell/`)

32. `unit/campbell/game.service.test.ts`

Path: [backend/tests/unit/campbell/game.service.test.ts](#) **Purpose:** Unit-tests `GameService` , the top-level orchestrator that handles two core operations: `startOrJoin` (connect a player and get or create their active route) and `completeRouteAndAdvance` (mark the current route done, pick the next one, and return

updated state). All six DB-layer dependencies are mocked with `vi.hoisted` factories so the service logic can be exercised in total isolation.

```
const { mockGetOrCreatePlayerSession, mockGetActiveRouteForPlayerSession,
      mockGetNextRouteForPlayerSession, mockCompleteActiveRouteForPlayerSession,
      mockCountCompletedRoutes, mockGetCompletedPollinators } = vi.hoisted(() =>
({ ... }));
```

Uses `vi.hoisted` so the mock factories run before any `import` statements. Each mock corresponds to one DB-layer service method: `getOrCreatePlayerSession`, `getActiveRouteForPlayerSession`, `getNextRouteForPlayerSession` (a transaction), `completeActiveRouteForPlayerSession`, `countCompletedRoutes`, and `getCompletedPollinators`.

```
vi.mock("../src/services/route.service", () => ({
  RouteService: class {
    completeActiveRouteForPlayerSession =
mockCompleteActiveRouteForPlayerSession;
    ...
  }
}));
```

`RouteService` is mocked as a class whose instance methods point at the hoisted mock functions — this matches how `GameService` constructs and calls `RouteService`.

```
const MOCK_PRISMA = {} as any;
const MOCK_PLAYER_SESSION = { playerId: 1, playerId: 5 };
const MOCK_ROUTE = { routeId: 1, name: "Route A", nodes: [...] };
function makeService() { return new GameService(MOCK_PRISMA); }
```

Shared test fixtures. `MOCK_PRISMA` is an empty object cast to `any` — `GameService` accepts a Prisma client via constructor injection but the mocks intercept all DB calls before they reach it.

`beforeEach(() => vi.clearAllMocks())` resets all mock call histories between tests.

startOrJoin tests

Test 1 — "returns playerId and existing active route when one is already active"

- Mocks `getOrCreatePlayerSession` → `MOCK_PLAYER_SESSION`,
`getActiveRouteForPlayerSession` → `MOCK_ROUTE`.
- Asserts `result.playerId === 5`, `result.activeRoute === MOCK_ROUTE`, and
`getNextRouteForPlayerSession` was **not** called (no new route allocation when one is already active).

Test 2 — "allocates a new route when no active route exists"

- `getActiveRouteForPlayerSession` → `null`, `getNextRouteForPlayerSession` →
`MOCK_ROUTE`.
- Asserts result contains `MOCK_ROUTE` and that `getNextRouteForPlayerSession` was called with
`{ sessionId: 1, playerId: 1, requireNodes: true }`.

Test 3 — "returns null activeRoute when no active route exists and none can be allocated"

- Both `getActiveRouteForPlayerSession` and `getNextRouteForPlayerSession` return `null`.
- Asserts `result.activeRoute` is `null` (no crash — absence of routes is a valid state).

Test 4 — "works without an optional playerId"

- Calls `service.startOrJoin(1)` (no second argument).
- Asserts `getOrCreatePlayerSession` was called with `(1, undefined)` and `result.playerId` comes from the player session, not the call argument.

Test 5 — "forwards sessionId and playerId to getOrCreatePlayerSession"

- Calls `service.startOrJoin(2, 5)`.
- Asserts `getOrCreatePlayerSession` was called with exactly `(2, 5)` — verifies argument threading.

Test 6 — "propagates errors thrown by getOrCreatePlayerSession"

- `getOrCreatePlayerSession` rejects with `new Error("DB error")`.
- Asserts `service.startOrJoin(1, 5)` rejects with the same message — errors are not swallowed.

completeRouteAndAdvance tests

Test 7 — "returns correct shape on success"

- All mocks return happy-path values. Calls `service.completeRouteAndAdvance(1, 5, 5000)`.
- Asserts the returned object has `playerId: 5`, `routesCompleted: 3`, `newActiveRoute: MOCK_ROUTE`, `completedPollinators: ["bee", "butterfly"]`.

Test 8 — "calls completeActiveRouteForPlayerSession with correct args"

- Calls `service.completeRouteAndAdvance(1, 5, 8000)`.
- Asserts `completeActiveRouteForPlayerSession` was called with `{ playerId: 1, completionMs: 8000 }`.

Test 9 — "passes requireNodes: true when fetching next route"

- Calls `service.completeRouteAndAdvance(1, 5)` (no `completionMs`).
- Asserts `getNextRouteForPlayerSession` was called with `{ sessionId: 1, playerId: 1, requireNodes: true }`.

Test 10 — "returns null newActiveRoute when no further routes exist"

- `getNextRouteForPlayerSession` → `null`.
- Asserts `result.newActiveRoute` is `null`.

Test 11 — "works without optional playerId and completionMs"

- Calls `service.completeRouteAndAdvance(1)` (only `sessionId`).
- Asserts `result.playerId` comes from the player session and `completeActiveRouteForPlayerSession` was called with `{ playerId: 10, completionMs: undefined }`.

Test 12 — "returns empty completedPollinators array when none exist"

- `getCompletedPollinators` → `[]` .
- Asserts `result.completedPollinators` is `[]` .

Test 13 — "propagates errors thrown by `completeActiveRouteForPlayerSession`"

- `completeActiveRouteForPlayerSession` rejects with `"complete failed"` .
- Asserts `completeRouteAndAdvance` rejects with the same message.

Test 14 — "propagates errors thrown by `getOrCreatePlayerSession`"

- `getOrCreatePlayerSession` rejects with `"session error"` .
- Asserts `completeRouteAndAdvance` rejects with that message (error surfaces before any route logic runs).

33. `unit/campbell/route.service.test.ts`

Path: `backend/tests/unit/campbell/route.service.test.ts` **Purpose:** Unit-tests `RouteService` , which owns the Prisma-level route and route-cycle operations. Six Prisma methods (`routeCycle.findFirst` , `routeCycle.update` , `routeCycle.count` , `routeCycle.findMany` , `route.count` , `route.findFirst`) are mocked via `vi.hoisted` . The `crypto` module is also mocked to control `randomInt` and make random-selection tests deterministic.

```
const { mockRouteCycleFindFirst, mockRouteCycleUpdate, ... } = vi.hoisted(() => ({
  ... }));
vi.mock("../../../src/services/db/prisma", () => ({ prisma: { routeCycle: {...},
route: {...} } }));
vi.mock("crypto", () => ({ default: { randomInt: vi.fn() } }));
```

Mocks the global `prisma` singleton and `crypto.randomInt` . The `MOCK_PRISMA` object passed to the `RouteService` constructor mirrors the same mock functions, so constructor-injected calls and module-level calls resolve to the same stubs.

`beforeEach(() => vi.clearAllMocks())` resets all mocks between tests.

`completeActiveRouteForPlayerSession` tests

Test 1 — "updates the active route cycle with `completedAt` and `wasCompleted`"

- `routeCycle.findFirst` → `{ id: 5 }` .
- Calls `completeActiveRouteForPlayerSession({ playerId: 1 })` .
- Asserts `routeCycle.update` was called with `where: { id: 5 }` and `data: { completedAt: <any Date>, completionMs: null, wasCompleted: true }` .

Test 2 — "includes `completionMs` when provided"

- Calls with `{ playerId: 1, completionMs: 4200 }` .
- Asserts the update data contains `completionMs: 4200` .

Test 3 — "sets `completionMs` to null when not provided"

- Calls with `{ playerId: 1 }` (no `completionMs`).
- Asserts the update data contains `completionMs: null` .

Test 4 — "queries for an incomplete cycle with a non-null `routeId`"

- Asserts `routeCycle.findFirst` was called with `where: { playerId: 1, completedAt: null, routeId: { not: null } }` — ensures the lookup targets only an in-progress cycle that has a route assigned.

Test 5 — "throws when no active route cycle exists"

- `routeCycle.findFirst` → `null`.
 - Asserts the call rejects with "No active route to complete" and `routeCycle.update` is never called.
-

countCompletedRoutes tests

Test 6 — "returns the count of completed route cycles"

- `routeCycle.count` → `5`.
- Asserts result is `5` and count was queried with `where: { playerId: 10, wasCompleted: true }`.

Test 7 — "returns 0 when no routes have been completed"

- `routeCycle.count` → `0`.
 - Asserts result is `0`.
-

getCompletedPollinators tests

Test 8 — "returns route names for completed cycles"

- `routeCycle.findMany` → `[{ route: { name: "bee" } }, { route: { name: "butterfly" } }]`.
- Asserts result is `["bee", "butterfly"]` and `findMany` was called with `where: { playerId: 10, wasCompleted: true }, select: { route: { select: { name: true } } }`.

Test 9 — "omits cycles where route is null"

- `findMany` returns `[{ route: { name: "bee" } }, { route: null }, { route: { name: "moth" } }]`.
- Asserts result is `["bee", "moth"]` — null routes (orphaned cycles) are filtered out.

Test 10 — "returns empty array when no completed cycles exist"

- `findMany` → `[]`.
 - Asserts result is `[]`.
-

getLatestCycleNumber tests

Test 11 — "returns cycleNumber from the most recent cycle"

- `routeCycle.findFirst` → `{ cycleNumber: 4 }`.
- Asserts result is `4` and `findFirst` was called with `where: { playerId: 10 }, orderBy: { cycleNumber: "desc" }, select: { cycleNumber: true }`.

Test 12 — "returns 1 when no cycle record exists"

- `routeCycle.findFirst` → `null`.
- Asserts result is `1` (the first cycle number is always 1 by convention).

pickRandomRouteId tests

Test 13 — "returns a route id when routes are available"

- `route.count` → 10 , `crypto.randomInt` → 3 , `route.findFirst` → { `id`: 5 } .
- Asserts result is 5 , `randomInt` was called with (0, 10) , and `findFirst` was called with `skip`: 3 (random offset selection).

Test 14 — "returns null when no routes match"

- `route.count` → 0 .
- Asserts result is null and `route.findFirst` is never called (skip the query when the pool is empty).

Test 15 — "returns null when findFirst returns nothing"

- `route.count` → 5 , `route.findFirst` → null .
- Asserts result is null .

Test 16 — "applies nodes filter when requireNodes is true"

- Asserts `route.count` was called with `where` containing `nodes`: { `some`: {} } — restricts the pool to routes that have at least one node.

Test 17 — "does not apply nodes filter when requireNodes is false"

- Asserts the `where` clause does not have a `nodes` key at all.

Test 18 — "applies excludeSeenFor filter when provided"

- Calls with `excludeSeenFor`: { `playerSessionId`: 10, `cycleNumber`: 2 } .
- Asserts `route.count` was called with `where` containing:

```
cycles: { none: { playerSessionId: 10, cycleNumber: 2, routeId: { not: null } } }
```

This excludes routes the player has already been assigned in the current cycle.

Test 19 — "does not apply excludeSeenFor filter when not provided"

- Asserts the `where` clause does not have a `cycles` key.

34. unit/campbell/reshape.utilities.test.ts

Path: [backend/tests/unit/campbell/reshape.utilities.test.ts](#) **Purpose:** Unit-tests

`ReshapeUtilities.mapRouteDTOToLegacyRoute` , a pure mapping function that converts the Prisma-style route DTO (numeric keys, `key` / `nodeId` fields) into the legacy shape the frontend expects (string IDs, `id` field, `routeId` / `routeNodes` / `factNodes` top-level keys). No mocks — entirely pure function tests.

```
import { describe, test, expect } from "vitest";
import { ReshapeUtilities } from "../../src/api/Utilities/reshape.Utilites";
```

Imports only the class under test. No mocking required.

The tests are organized into four `describe` blocks:

routeId block

Test 1 — "converts a numeric route id to a string"

- Input: `{ id: 7, nodes: [], facts: [] }`.
- Asserts `result.routeId === "7"` — numeric IDs must be stringified for the legacy frontend.

Test 2 — "preserves a route id that is already a string"

- Input: `{ id: "42", nodes: [], facts: [] }`.
- Asserts `result.routeId === "42"` — no double-conversion.

Test 3 — "converts a large numeric route id correctly"

- Input: `{ id: 1_000_000, nodes: [], facts: [] }`.
- Asserts `result.routeId === "1000000"`.

routeNodes block

Test 4 — "returns an empty array when nodes is an empty array"

- Input: `nodes: []`. Asserts `result.routeNodes` is `[]`.

Test 5 — "returns an empty array when nodes is undefined"

- Input: no `nodes` key. Asserts `result.routeNodes` is `[]`.

Test 6 — "maps a single node's key to id as a string"

- Input: `nodes: [{ key: 10, text: "Oak tree" }]`.
- Asserts `result.routeNodes[0].id === "10"` — `key` → `id` rename + stringify.

Test 7 — "maps a single node's text field"

- Asserts `result.routeNodes[0].text === "Oak tree"` — text is passed through unchanged.

Test 8 — "maps multiple nodes preserving order"

- Input: 3 nodes. Asserts output is `[{ id: "1", text: "Stop A" }, { id: "2", text: "Stop B" }, { id: "3", text: "Stop C" }]`.

Test 9 — "converts a numeric key to a string id"

- `key: 99` → `typeof result.routeNodes[0].id === "string"`.

Test 10 — "converts a numeric text value to a string"

- `text: 123` → `result.routeNodes[0].text === "123"`.

factNodes block

Test 11 — "returns an empty array when facts is an empty array"

- Input: `facts: []`. Asserts `result.factNodes` is `[]`.

Test 12 — "returns an empty array when facts is undefined"

- Input: no `facts` key. Asserts `result.factNodes` is `[]`.

Test 13 — "maps a single fact's nodeId to id as a string"

- Input: `facts: [{ nodeId: 5, text: "Bees love lavender", pollinator: "bee" }]` .
- Asserts `result.factNodes[0].id === "5"` — `nodeId` → `id` rename + stringify.

Test 14 — "maps a single fact's text field"

- Asserts `result.factNodes[0].text === "Bees love lavender"` .

Test 15 — "maps a single fact's pollinator field"

- Asserts `result.factNodes[0].pollinator === "bee"` .

Test 16 — "maps multiple facts preserving order"

- Input: 3 facts. Asserts full `factNodes` array with correct `id` , `text` , `pollinator` for each.

Test 17 — "converts a numeric nodeId to a string id"

- `nodeId: 77` → `typeof result.factNodes[0].id === "string"` .

Test 18 — "converts a numeric pollinator value to a string"

- `pollinator: 42` → `result.factNodes[0].pollinator === "42"` .

full route shape block

Test 19 — "returns the correct top-level shape with all three fields"

- Asserts output has properties `routeId` , `routeNodes` , `factNodes` — validates the contract shape.

Test 20 — "maps a complete route DTO with both nodes and facts correctly"

- Input: `{ id: 3, nodes: [{ key: 10, "Meadow" }, { key: 11, "Pond" }], facts: [{ nodeId: 10, ... }, { nodeId: 11, ... } ] }` .
- Asserts the full deeply-equal output — integration check across all three fields simultaneously.

Test 21 — "handles a route with nodes but no facts"

- `facts: []` → `result.factNodes` is `[]` , `result.routeNodes.length === 1` .

Test 22 — "handles a route with facts but no nodes"

- `nodes: []` → `result.routeNodes` is `[]` , `result.factNodes.length === 1` .

34b. getNextRouteForPlayerSession.service.test.ts

Path: [backend/tests/unit/campbell/getNextRouteForPlayerSession.service.test.ts](#) **Purpose:** Unit-tests the `getNextRouteForPlayerSessionService.getNextRouteForPlayerSession()` transaction service — the core route-allocation logic that guarantees players see all routes before repeating any. Tests cover mid-cycle picks, cycle-boundary transitions, exclusion fallbacks, concurrency retries, and the retry-limit guard.

All dependencies are mocked via `vi.hoisted` :

- `mockGetLatestCycleNumber` / `mockPickRandomRouteId` — from `route.service`
- `mockRouteCycleFindMany` / `mockRouteCycleCreate` — Prisma transaction client
- `mockPrismaTransaction` — intercepts `prisma.$transaction` to call the callback synchronously with `mockTx`
- `mockLoadRouteDTO` — from `LoadRouteDTOService`

Constants used across all tests:

```
const INPUT = { sessionId: 10, playerSessionId: 1 };
const MOCK_ROUTE_DTO = { id: 5, name: "Fly", routeNodes: [], factNodes: [] };
```

Mid-cycle group (3 tests)

Test — "mid-cycle: returns a route from the current cycle when one is available"

- `getLatestCycleNumber` → 1 ; `pickRandomRouteId` → 5 .
- Asserts `result` equals `MOCK_ROUTE_DTO` and `loadRouteDTO` was called with 5 .

Test — "mid-cycle: creates a RouteCycle row with the current cycleNumber"

- `getLatestCycleNumber` → 3 ; `pickRandomRouteId` → 7 .
- Asserts `routeCycle.create` was called with data containing { `routeId`: 7, `cycleNumber`: 3, `sessionId`: 10, `playerSessionId`: 1, `wasCompleted`: false } .

Test — "mid-cycle: passes excludeSeenFor with the current cycleNumber to pickRandomRouteId"

- `getLatestCycleNumber` → 2 .
- Asserts `pickRandomRouteId` was called with `expect.objectContaining({ excludeSeenFor: { playerSessionId: 1, cycleNumber: 2 } })` .

Cycle boundary group (5 tests)

Test — "cycle boundary: starts a new cycle when no unseen routes remain"

- `pickRandomRouteId` returns `null` on the first call (all routes in cycle 1 seen), then 6 on the second call.
- Asserts `routeCycle.create` was called with { `cycleNumber`: 2 } — the cycle incremented.

Test — "cycle boundary: excludes the 2 most recently completed routes when starting the new cycle"

- `routeCycleFindMany` returns [{ `routeId`: 5 }, { `routeId`: 3 }] .
- Asserts the second `pickRandomRouteId` call includes `excludeRouteIds`: [5, 3] .

Test — "cycle boundary: queries recently completed routes ordered by completedAt desc, limit 2"

- Asserts `routeCycleFindMany` was called with:

```
{
  where: { playerSessionId: 1, wasCompleted: true },
  orderBy: { completedAt: "desc" },
  take: 2,
  select: { routeId: true },
}
```

Test — "cycle boundary: handles only 1 recently completed route (excludes that one id)"

- `routeCycleFindMany` returns [{ `routeId`: 5 }] .

- Asserts `pickRandomRouteId` second call has `excludeRouteIds: [5]` (only one exclusion).

Test — "cycle boundary: uses no exclusions when no completed routes exist yet"

- `routeCycleFindMany` returns `[]`.
 - Asserts the second `pickRandomRouteId` call does NOT have an `excludeRouteIds` property.
-

Fallback group (1 test)

Test — "fallback: picks any route when all routes are in the exclusion list"

- `pickRandomRouteId` returns `null` twice (no unseen routes, then none after exclusions), then `5` on the third call.
 - Asserts final result equals `MOCK_ROUTE_DTO` and the third `pickRandomRouteId` call has no `excludeRouteIds` or `excludeSeenFor`.
-

No routes group (1 test)

Test — "returns null when no routes exist at all"

- `pickRandomRouteId` always returns `null`; `routeCycleFindMany` returns `[]`.
 - Asserts result is `null` and `routeCycle.create` was never called.
-

Error handling group (2 tests)

Test — "retries on P2002 conflict and returns the route on the next attempt"

- First `$transaction` call rejects with `{ code: "P2002" }`; second succeeds via the default implementation.
- Asserts `$transaction` was called twice and result equals `MOCK_ROUTE_DTO`.

Test — "rethrows non-P2002 transaction errors"

- `$transaction` always rejects with `new Error("DB connection lost")`.
 - Asserts the service rejects with the same error — non-P2002 errors are not retried.
-

Retry limit group (1 test)

Test — "throws after exhausting retries when the route vanishes after every selection"

- `pickRandomRouteId` always returns `5`; `loadRouteDTO` always returns `null` (route disappears between selection and load).
 - Asserts rejects with `"Failed to allocate next route after retries"`.
 - Has a custom 15 000 ms timeout because the retry loop runs 300 iterations.
-

Portal UAT Tests

35. `UAT/ExportSearchResults.test.tsx`

Path: [portal/testing/UAT/ExportSearchResults.test.tsx](#) **Purpose:** Smoke test that confirms the portal's main page (`PortalPage`) renders without crashing. This is a placeholder-level UAT test — no assertions beyond a successful render.

```
import { render, screen, waitFor } from '@testing-library/react';
import { describe, it, expect } from 'vitest';
import PortalPage from '../../src/app/page';
```

Imports RTL helpers and the portal root page component.

```
describe('ExportSearchResults', () => {
  it('should render the export search results page', async () => {
    render(<PortalPage />);
  });
});
```

Renders `PortalPage` and passes if no exception is thrown. No `screen` queries or `expect` assertions are present — the test validates the component tree mounts without errors. This is intentionally minimal; full functional coverage is expected to be added as the export feature is built out.

Document generated for commit reference — not tracked by git.